Laboratory Exercise 5
Multiplexers, Comparators and a Simple ALU

Introduction

This laboratory exercise accompanies the second half of Lecture 5 (Combinational Circuit Building Blocks): multiplexers, Boolean function implementation with a multiplexer, tri-state gates, the Arithmetic Logic Unit (ALU) (and, as an extension, the magnitude comparator). The circuits are written in VHDL. You will build a 4-to-1 multiplexer from 2-to-1 multiplexers like the one that you built on the breadboard in Lab 5.5, check it with a self-checking **testbench**, and then combine the building blocks of Labs 3 to 5 into a small ALU. The ALU is described with a first VHDL `PROCESS`, which prepares you for the sequential circuits in Lab 6.

Objectives

1. To build a larger multiplexer from smaller multiplexers by structural VHDL, and to compare it with a behavioral description.
2. To verify a design exhaustively with a self-checking VHDL testbench.
3. To implement a Boolean function with a multiplexer.
4. (Extension) To design a 4-bit magnitude comparator.
5. To describe a combinational circuit with a `PROCESS` and a `CASE` statement, and to use a tri-state output.

Equipment Required

- Intel DE0-CV board and a PC with Quartus Prime (and the simulator selected by your instructor)
- Files from previous labs: `hex7seg.vhd` (Lab 4)

Pre-lab (complete before the session)

The session is two hours long, so complete the following *before* the session.

1. Draw a 4-to-1 multiplexer built with three 2-to-1 multiplexers. Which select signal drives the first stage and which the second stage?
2. For $F(A, B, C) = \sum m(1, 3, 5, 6, 7)$ fill in the inputs of a 4-to-1 multiplexer with A, B on the select inputs ($S_1 = A$, $S_0 = B$): $I_0 =$ _____, $I_1 =$ _____, $I_2 =$ _____, $I_3 =$ _____.

Part I: Multiplexers and a Testbench (about 40 min)

2-to-1 multiplexer

The 2-to-1 multiplexer with $Y = \bar{S}\, I_0 + S\, I_1$ is the circuit of Lab 5.5. In VHDL, the `WHEN ... ELSE` conditional assignment describes the same behavior in one line.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux2to1 IS
    PORT ( I0, I1, S : IN  STD_LOGIC;
           Y         : OUT STD_LOGIC );
END mux2to1;

ARCHITECTURE behavior OF mux2to1 IS
BEGIN
    Y <= I0 WHEN S = '0' ELSE I1;
END behavior;
```

Figure 1: A 2-to-1 multiplexer.

4-to-1 multiplexer in two ways

1. Create a new Quartus project `lab5` and enter `mux2to1.vhd` (Figure 1).
2. Write `mux4_struct`: a 4-to-1 multiplexer with inputs $I(3..0)$, $S(1..0)$ and output Y using **three instances** of `mux2to1`. Use S_0 in the first stage and S_1 in the second stage (see Pre-lab). You need two internal signals for the outputs of the first stage.
3. Write `mux4_behav`: the same multiplexer described with a selected signal assignment:

```vhdl
WITH S SELECT
    Y <= I(0) WHEN "00",
         I(1) WHEN "01",
         I(2) WHEN "10",
         I(3) WHEN OTHERS;
```

4. Compile both designs separately (set each as the top-level entity) and record the number of logic elements and the RTL Viewer picture. Are they the same?

A self-checking testbench

A testbench is a VHDL entity without ports that instantiates the design under test, applies inputs, and checks the outputs. The testbench in Figure 2 applies all $2^6 = 64$ combinations of I and S to both multiplexers and reports an error if the outputs differ. It uses a `PROCESS`, a statement whose body is executed *in sequence*, with `WAIT FOR` to let time pass.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY mux4_tb IS END mux4_tb;

ARCHITECTURE sim OF mux4_tb IS
    SIGNAL I          : STD_LOGIC_VECTOR(3 DOWNTO 0);
    SIGNAL S          : STD_LOGIC_VECTOR(1 DOWNTO 0);
    SIGNAL Y_s, Y_b   : STD_LOGIC;
BEGIN
    u1 : ENTITY work.mux4_struct PORT MAP (I => I, S => S, Y => Y_s);
    u2 : ENTITY work.mux4_behav  PORT MAP (I => I, S => S, Y => Y_b);

    stimulus : PROCESS
        VARIABLE v : UNSIGNED(5 DOWNTO 0);
    BEGIN
        FOR k IN 0 TO 63 LOOP
            v := TO_UNSIGNED(k, 6);
            I <= STD_LOGIC_VECTOR(v(5 DOWNTO 2));
            S <= STD_LOGIC_VECTOR(v(1 DOWNTO 0));
            WAIT FOR 10 ns;
            ASSERT Y_s = Y_b REPORT "Mismatch at k = " & INTEGER'IMAGE(k)
                SEVERITY ERROR;
        END LOOP;
        REPORT "Test finished";
        WAIT;
    END PROCESS;
END sim;
```

Figure 2: Exhaustive self-checking testbench for the two multiplexers.

5. Run the simulation of `mux4_tb` (the simulator and its set-up are explained by your instructor) for at least 640 ns. Are there any error messages? Show the waveform of I, S, Y_s and Y_b.

6. Connect `mux4_behav` to the board: I = SW3–SW0, S = SW9–SW8, Y = LEDR0. Test all select values.

Part II: Implementing a Function with a Multiplexer (about 20 min)

A 2^n-to-1 multiplexer can implement any function of $n+1$ variables (Shannon's expansion, Lecture 5). Use your `mux4_behav` to implement $F(A,B,C) = \sum m(1,3,5,6,7)$.

1. Write an entity `func_mux` with inputs A, B, C and output F. Connect A to S_1, B to S_0 and the inputs $I_3..I_0$ as you found in the Pre-lab (the constants are `'0'` and `'1'`, and signal C for the others).

2. Write a second entity `func_sop` that implements F directly as a minimized SOP expression. Use a K-map to find the expression: $F =$ _____

3. Simulate both and show that they are equivalent. Compile each design separately: which uses fewer logic elements? _____

4. Implement `func_mux` on the board with A, B, C = SW2–SW0 and F = LEDR0 and complete Table 1.

A	B	C	F expected	F measured
0	0	0	0	
0	0	1	1	
0	1	0	0	
0	1	1	1	
1	0	0	0	
1	0	1	1	
1	1	0	1	
1	1	1	1	

Table 1: Function implemented with a multiplexer.

Part III: A Simple ALU with Tri-State Output (about 35 min)

An ALU performs several arithmetic or logic operations on two operands; a multiplexer selects which result appears at the output. Design a 4-bit ALU with the operations of Table 2 (Figure 3). Use a `PROCESS` with a `CASE` statement for the multiplexing. In a process, signals read inside the process must appear in its **sensitivity list** (the list in parentheses after `PROCESS`).

Op (2 bits)	**Function**
00	$Y = A + B$
01	$Y = A - B$
10	$Y = A$ AND B
11	$Y = A$ OR B

Table 2: ALU operations.

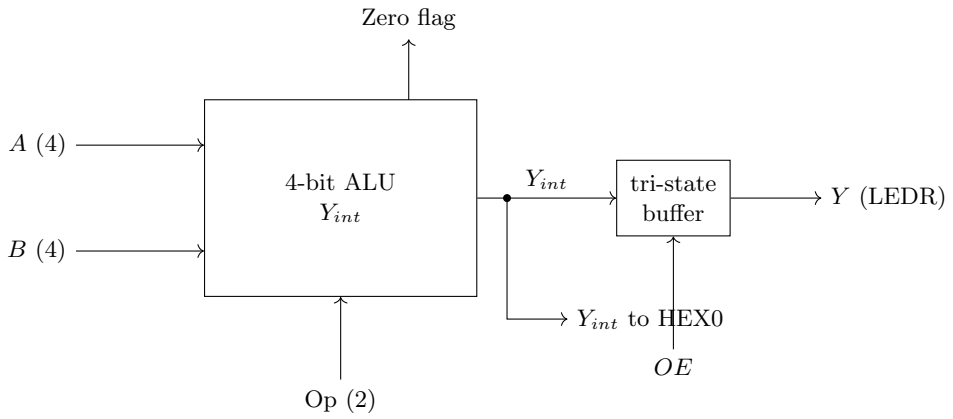


Figure 3: Block diagram of the ALU with a tri-state output stage.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY alu4 IS
    PORT ( A, B  : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
           Op    : IN  STD_LOGIC_VECTOR(1 DOWNTO 0);
           OE    : IN  STD_LOGIC;
           Yint  : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);   -- always driven
           Y     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);   -- tri-state output
           Zero  : OUT STD_LOGIC );
END alu4;

ARCHITECTURE behavior OF alu4 IS
    SIGNAL R : STD_LOGIC_VECTOR(3 DOWNTO 0);
BEGIN
    PROCESS (A, B, Op)
    BEGIN
        CASE Op IS
            WHEN "00"   => R <= STD_LOGIC_VECTOR(UNSIGNED(A) + UNSIGNED(B));
            WHEN "01"   => R <= STD_LOGIC_VECTOR(UNSIGNED(A) - UNSIGNED(B));
            WHEN "10"   => R <= A AND B;
            WHEN OTHERS => R <= A OR B;
        END CASE;
    END PROCESS;

    Yint <= R;
    Zero <= '1' WHEN R = "0000" ELSE '0';
    Y    <= R WHEN OE = '1' ELSE "ZZZZ";     -- tri-state buffer
END behavior;
```

Figure 4: A 4-bit ALU with zero flag and tri-state output.

1. Add `alu4.vhd` to the project and compile it. Why can the output ports `Yint` and `Y` not be read inside the architecture, and why do we use the internal signal `R`?

2. Simulate the ALU. Choose test vectors that include the following cases and record the results in Table 3: $9 + 8$ (the result does not fit in 4 bits), $3 - 5$, and $A - A$ (the Zero flag).

3. Write the top-level entity: A = SW3–SW0, B = SW7–SW4, Op = SW9–SW8, Y = LEDR3–LEDR0, `Zero` = LEDR9, and `Yint` displayed on `HEX0` with `hex7seg`. Use push-button KEY0 as the output enable: `OE <= NOT KEY(0);` (KEY is active-low, so the outputs are enabled *while the button is pressed*).

4. Download to the board and test Table 3. What do the LEDs display when KEY0 is not pressed? Why is this output state called "high impedance"?

5. Tri-state outputs allow several circuits to share one wire (a bus), provided that only one is enabled at any time. *Inside* an FPGA chip no tri-state buses exist; the synthesis tool converts the tri-state buffers into multiplexers, except at the chip pins. Why can two ordinary gate outputs not be connected together without tri-state control? (Lecture 5, Exercise 5-3.)

Op	A	B	Y_{int} expected	HEX0	Zero	LEDR3–0 (KEY0 pressed)
00	9	8				
01	3	5				
01	7	7				
10	1100	1010				
11	1100	1010				

Table 3: ALU test results.

Extension (take-home, or if time permits)

1. (Part I) To see the effect of a failing test, change one line of `mux4_struct` deliberately (for example, swap S_0 and S_1) and rerun the testbench. What message appears? Restore the correct code.

4-bit Magnitude Comparator

A magnitude comparator compares two numbers A and B. For 4 bits, define the *equality* signals $x_i = A_i \odot B_i$ (XNOR). Then

$$(A = B) = x_3\, x_2\, x_1\, x_0$$
$$(A > B) = A_3\bar{B}_3 + x_3A_2\bar{B}_2 + x_3x_2A_1\bar{B}_1 + x_3x_2x_1A_0\bar{B}_0$$
$$(A < B) = \bar{A}_3B_3 + x_3\bar{A}_2B_2 + x_3x_2\bar{A}_1B_1 + x_3x_2x_1\bar{A}_0B_0$$

1. Write an entity `comp4_eq` that uses these equations with one `SIGNAL x : STD_LOGIC_VECTOR(3 DOWNTO 0)` for the equality signals, e.g. `x <= A XNOR B;`. The ports are A, B (4 bits) and `AGB`, `AEB`, `ALB`.

2. Write a second entity `comp4_beh` with the same ports, but using the relational operators on unsigned numbers:

```
USE ieee.numeric_std.ALL;
...
AGB <= '1' WHEN UNSIGNED(A) >  UNSIGNED(B) ELSE '0';
```

3. Write a testbench (modify Figure 2: loop over 0..255, split v into A and B) that compares both comparators for all 256 input combinations.

4. On the board: A = SW3–SW0, B = SW7–SW4, `AGB`, `AEB`, `ALB` on LEDR2, LEDR1, LEDR0. Also display A on `HEX1` and B on `HEX0` (with `hex7seg`) and check the three cases of Table 4.

A	B	AGB	AEB	ALB
1011	1001			
0110	0110			
0011	1000			

Table 4: Comparator results.

Check-off

Show your instructor or teaching assistant:

1. The testbench waveform of the multiplexers without errors.
2. The function implemented with a multiplexer on the board.
3. The ALU with tri-state output working on the board (Table 3).

Updated By: R. Sutthaweekul
Version: 1.0
Release Date: 2026-11-11